

Haoran Feng

Phone: (+86) 15024999885 Email: 225010160@link.cuhk.edu.cn

EDUCATION

Sep 2025–Jun 2027 (Expected)
Sep 2021–Jun 2025

The Chinese University of Hong Kong, Shenzhen
Shandong University (Project 985)

M.S. in Integrated Circuits and Systems
B.S. in Electronic Science and
Technology / Computer Science and
Technology

Honors: Academic Scholarship (Top 20%); Special Talent Scholarship for Innovation and Competitions.

EXPERIENCE

Apr 2026–Present

Ubiquant Investment — AI Research Institute

LLM Systems & High-Performance
Computing Intern

- **FlashInfer CUDA Graph Stability:** Resolved vLLM rollout hangs under TP=2 FULL CUDA Graph on H200. Built a two-GPU reproducer and traced the issue to the polling logic in fused AllReduce + RMSNorm: FTZ caused the floating-point check for a Lamport -0.0 sentinel to misclassify valid negative FP32 subnormals, leaving the GPU kernel spinning indefinitely. Replaced the check with an exact 0x80000000 bit-pattern comparison; the two-GPU reproducer and model-level Graph on/off regressions all passed, with no timeout or hang across repeated Graph replays.
- **R3 Router Replay:** Reproduced Xiaomi's published R3 method to address MoE routing divergence between vLLM rollout and Megatron training. Implemented route capture, transport, and replay at [token, MoE layer, top-k] granularity, plus response-mask alignment and explicit anomaly checks; covered 18 CPU tests. In a 20-step run on 8×H200 with Qwen3-30B-A3B BF16, reduced route mismatch from 17%–19% to 0, F_{τ_2} by 36–145×, and KL by 4–7×. Scaled to 128 GPUs, completed a 200-step single-round competition task with an internal 300B model under SwanLab monitoring; results matched the 8-GPU run and the changes were merged into the development branch.
- **Deterministic Router GEMM Optimization on H200:** Eliminated wasted work from vLLM Batch-Invariant mode's fixed 128×128×64 persistent tile in small-M decode by designing and integrating deterministic Triton full-K and DeepGEMM SM90 BF16 paths, validated from kernel and CUDA Graph to full-model execution. For BF16 [M, 2048]×[2048, 128] with M=1/2/4/8/12, Triton full-K was 1.99–2.15× faster than the existing BI kernel, while DeepGEMM reduced Graph p50 from 13.79–14.94 μs to 4.74–4.86 μs (2.84–3.15×). Across 20 target-model scenarios, median 48-layer Router GEMM time fell from 789.23 to 210.01 μs/step (−73.39%) and median per-step GPU busy time by 6.16%, with bitwise agreement in 135/135 kernel and 20/20 full-model tests. Delivered a three-tier backend: DeepGEMM primary, Triton full-K low-dependency fallback, and persistent full-coverage fallback for M>2048 or unsupported configurations.
- **OE Asynchronous Operator Development:** Kept recent-token history and oe_input_ids construction on GPU and used a Triton fused hash to remove GPU-to-CPU synchronization waits and CPU-to-GPU transfers. Fixed cross-step state updates across prefill, decode, mixed batches, request resumption, slot reuse, and reorder. All 28 TP1 cases passed; throughput improved 4.99% over sync and 1.94% over async-unfused. For TP2/TP4, a batch-invariant safety path raised deterministic pass counts from 41/84 and 63/84 to 84/84 with zero mismatches, while decode throughput improved ~4.6%/~3.0%.

Jan 2026–Apr 2026

Moore Threads

Operator & Compiler Optimization
Intern

- **TensorFlow MUSA Extension:** Delivered operator enablement, plugin integration, validation, and graph optimization for TensorFlow on MUSA. Integrated muDNN GELU, repaired the GELU fusion path, and built benchmarks, enabling all 11 GELU nodes in the target graph to fuse and reducing runtime on production shapes by ~36.6%.
- **OOM & Crash Resolution:** Resolved OOM and intermittent crashes in long-running full-model inference. Traced shape tensors in StridedSlice<int32>/Pack<int32> being dispatched to the device path, rebuilt HostMemory handling, improved success from ~30% over 500 iterations to 100% over 1,000, then passed 40K/400K/800K stress iterations.
- **Hot-Operator Optimization & Stability:** Optimized the Logical_Or scalar-broadcast path and added Logical_And tests, cutting average latency from 21.2 μs to 10.7 μs and raising full-model throughput from 8,187.48 to 8,284.65 under the same configuration; also optimized the AddV2 hot path and aligned ResourceApplyAdam results in graph mode.

PROJECTS

Dec 2025–Present

High-Performance Quantized LLM
Inference Runtime via PyTorch
Extensions

- **Quantized Runtime & Backend Integration:** Extended a PyTorch Extension runtime supporting W4A16 AWQ, W8A8 SmoothQuant, and FP8.
Integrated configurable FlashAttention-2/4; added GQA KV-head expansion, softcap masking, and SDPA-to-Torch fallback; made CUDA architecture, ccache, and model paths configurable for builds on a local RTX 5060 and H200.
- **H200 Kernels & Tensor Parallelism:** Ported the AWQ runtime to 2×H200 (SM90/CUDA 13.0). Using NVIDIA Compute Sanitizer, traced an illegal decode access in `gemv_kernel_g128` to a warp reading eight groups when `K/group_size=7`, overrunning scale/zero in trailing lanes. Added group-bound checks, current-stream launches, and error checks; representative shapes passed memcheck/initcheck/synccheck/racecheck with 0 errors/0 hazards. Implemented Qwen2 packed-AWQ TP=2 with column parallelism for Q/K/V and gate/up, row parallelism for O/down, and NCCL all-reduce.
- **32B Quantized Deployment & Validation:** Quantized and delivered Qwen2.5-Coder-32B W4A16 on two GPUs using 128×512 local-code calibration. Under the same model, prompt, 64-token output, and three repeats, reduced checkpoint size from 61.04 to 18.02 GiB (−70.5%), increased E2E output throughput from 2.53 to 4.47 tok/s (+76.8%), and cut peak runtime memory from 32.02 to 10.63 GiB/rank (−66.8%). Passed 10 CUDA/PyTorch numerical comparisons (max absolute error 0.007812), cross-rank token-consistency checks, and Chinese-language interactive acceptance tests.

TECHNICAL SKILLS

- **Technical Focus:** RL infrastructure, GPU operator and performance optimization, and LLM inference systems; hands-on experience with MoE train–inference consistency, multi-GPU inference, and quantized runtimes.
- **Programming & Performance:** C/C++, Python, CUDA, Triton, and PyTorch Extensions; engineering experience with PyTorch, veRL, Megatron-LM, vLLM, FlashInfer, CUTLASS, and NCCL. Use Nsight Systems (Nsys), Nsight Compute (NCU), and NVIDIA Compute Sanitizer for kernel profiling, fault isolation, multi-GPU correctness validation, and Kernel–Operator–E2E performance regression.
- **Engineering & Languages:** Cross-repository source analysis, minimal reproducers, controlled experiments, automated testing, benchmarking, and technical documentation. Use Claude Code and Codex for code search, implementation, and testing, with human review and correctness/performance regressions. English: IELTS 6.5, CET-6; Huawei HCIA-AI certified.